19. Gaussian Naive Bayes

Still a classification model based on the Bayes' theorem. But it is born for continunous values

Previous Section:

18. Naive Bayes Classifier

Contents:

This section is about Gaussian Naive Bayes. It is still a classification model. It is still built on Bayes' Theorem. Nothing fundamentally changes at the core.

But the moment your data stops being clean categories, and starts becoming real numbers. This is where Gaussian Naive Bayes begins to exist.

Up until now, Naive Bayes felt simple. You counted things. Frequencies. Occurrences: *"Word appears 3 times in spam, 1 time in not spam."*

Everything was discrete. Countable. Almost tangible. But reality doesn't always give you that comfort. What if your feature is not "Marketing" or "Transactional", but something like:

- $Age = 23.5$
- $Salary = 72,300$
- $Temperature = 36.7$

You can't count these the same way. You won't see the exact same value repeating nicely across the dataset. And this is the breaking point of the basic Naive Bayes idea.

So instead of asking: "How many times did this exact value appear?" → We shift the question into something more continuous. *"How likely is this value... given the class?"*

That's the key transition. Gaussian Naive Bayes doesn't count occurrences. It measures likelihood using a distribution. And not just any distribution. It assumes that your data follows a **Gaussian (Normal) distribution**.

Which means: Instead of storing counts, the model learns two things for each feature and each class:

- The **mean** (where the data is centered)
- The **variance** (how spread out the data is)

And from those two numbers, it can estimate the probability of *any* continuous value.

Even values it has never seen before. That's the power here. It doesn't memorize exact values. It understands the shape of the data.

So yes, Gaussian Naive Bayes is still Naive Bayes. → Still independent features. Still probability multiplication. Still classification. But now... instead of counting discrete events... it models continuous reality.

And that one shift... is what makes it usable in the real world.

1. Gaussian Naive Bayes - Naive Bayes for continuous features

Gaussian Naive Bayes is a type of Naive Bayes model that handles continuous features. Typically, these features follow a Gaussian Distribution.

And honestly... this is where things start to feel a bit more real. Categorical Naive Bayes was comfortable. Everything was clean. Discrete. Countable. Words, labels, categories, things you could just tally up.

But the world doesn't always come in categories. Height. Weight. Temperature. Measurements. They don't repeat nicely. They don't sit in buckets. They just exist on a spectrum.

So the question changes. We're no longer asking: "How many times did this value appear?". Instead, we ask something more subtle: *"How likely is this value... given the class?"*

That's where Gaussian Naive Bayes steps in. Gaussian Naive Bayes is essentially an extension of Categorical Naive Bayes, but designed for numerical (continuous) features.

Instead of counting frequencies, it assumes something about the data: Within each class, each feature follows a **Gaussian (normal) distribution**. Not perfectly. Not always. But close enough for the model to work surprisingly well.

So rather than storing counts, the model learns the *shape* of the data. For every feature in every class, it estimates:

- A **mean (μ)** → where the data is centered
- A **standard deviation (σ)** → how spread out it is

And from just these two values, it can describe the entire distribution.

To compute probabilities, Gaussian Naive Bayes uses the Gaussian probability density function:

$$P(x_i \mid y) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

This looks intimidating at first glance. But don't overthink it. It's just a way of saying:

> *"If the data is centered at μ, and spreads with σ... how likely is it to observe this value x?"*

That's it. But here is a catch: There *is* another way to deal with continuous data. You could force it back into categories. Split values into ranges:

- Age : 0–$20, 20$–$40, 40$–60
- $Salary$: $Low, Medium, High$

This process is called **binning**. And it works. But it comes with a cost. Because the moment you create those bins, you're making decisions. Arbitrary ones. Where do you split? Why 20? Why not 25? And those decisions can distort reality. You might lose important patterns. Flatten subtle differences. Or worse, introduce bias that wasn't even there.

Gaussian Naive Bayes avoids all of that. It doesn't force the data into boxes. It doesn't simplify the world just to make computation easier. Instead, it accepts the data as it is. Continuous. Messy. Natural. By modeling features with a normal distribution, it preserves more information, produces smoother probability estimates, and often leads to better classification performance.

So in the end, Gaussian Naive Bayes is still Naive Bayes. Still simple. Still probabilistic. Still "naive" in its independence assumption. But now, it stops counting. And starts *understanding the shape of data*.

2. Gaussian Naive Bayes - The Workflow

We will demonstrate Gaussian Naive Bayes with this dataset:

Email Size (KB)	Link Count	Unique Word Frequency (%)	Email Length (Chars)	Target (Status)
48.97	2	65.2	1240	**Promotion**
14.59	11	22.1	410	**Spam**
28.24	1	82.4	850	**Not Spam**
32.62	2	79.8	920	**Not Spam**
23.83	1	88.5	710	**Not Spam**
14.30	9	18.4	380	**Spam**
32.90	3	75.2	980	**Not Spam**
51.14	5	61.9	1310	**Promotion**
22.65	2	84.1	690	**Not Spam**
27.71	1	81.3	820	**Not Spam**
41.29	4	58.7	1150	**Promotion**
22.67	2	86.4	680	**Not Spam**
26.21	1	83.2	790	**Not Spam**
9.26	13	15.6	320	**Spam**

Now look at this carefully. At first glance, it might seem like we could still use the normal Naive Bayes. After all, some values *look* discrete:

- Link Count → $1, 2, 3, 4 \ldots$ You *could* treat each number as a category

Technically, you can. But this is exactly where things start to feel wrong.

Think about it. If you treat $\text{Link Count} = 1$ and $\text{Link Count} = 2$ as completely different categories, you're saying: "There is no relationship between them."

Which doesn't make sense. Because in reality:

- ***1 link*** and ***2 links*** are *very similar*
- ***2 links*** and ***13 links*** are *very different*

But Categorical Naive Bayes doesn't understand that. To it, everything is just a label. No distance. No scale. No meaning between numbers.

And it gets worse for the other features:

- $\text{Email Size} = 48.97$ vs 49.01 → treated as completely different
- Unique $\text{Word Frequency} = 82.4$ vs 82.5 → completely unrelated

Even though they are almost identical. So what happens? You end up with too many "unique categories", very sparse counts, probabilities that don't generalize well → The model starts to break... quietly.

This is exactly why Gaussian Naive Bayes exists. Instead of forcing these values into fake categories, it treats them the way they are meant to be treated: As **continuous signals**.

It understands that:

- **80%** and **82%** are close
- **300 chars** and **320 chars** are close
- **1 link** and **2 links** are close

Not identical, but related. So rather than counting how many times "**48.97 KB**" appears. It asks: ***"Given the class is Spam... what kind of range of email sizes do we usually see?"***

And that's a completely different mindset. Even for features that *look* discrete (like **Link Count**), Gaussian Naive Bayes still works better in many cases. Because it doesn't just see numbers, it sees **distribution**.

- Spam emails → tend to have *high* link counts

- Not Spam → usually *low*
- Promotion → somewhere in between

That pattern is not about exact values. It's about shape.

Overall, you *can* force this dataset into Categorical Naive Bayes. But you'd be throwing away something important. The relationships between numbers. The continuity of the data. The subtle structure hiding inside.

Gaussian Naive Bayes doesn't ignore that structure. It leans into it. And that's exactly why this dataset is a perfect place to use it.

- Step 1: **Separate by Class**

We separate the data into three classes: $Spam, NotSpam, Promotion$. Each one is its own behavior, its own pattern, its own "personality".

```
email_data = {"Email Size (KB)": [48.97, 14.59, 28.24, 32.62, 23.83, 14.30, 32.90,
51.14, 22.65, 27.71, 41.29, 22.67, 26.21, 9.26],
              "Link Count": [2, 11, 1, 2, 1, 9, 3, 5, 2, 1, 4, 2, 1, 13],
              "Unique Word Frequency (%)": [65.2, 22.1, 82.4, 79.8, 88.5, 18.4, 75.
2, 61.9, 84.1, 81.3, 58.7, 86.4, 83.2, 15.6],
              "Email Length (Chars)": [1240, 410, 850, 920, 710, 380, 980, 1310, 69
0, 820, 1150, 680, 790, 320],
              "Target": ["Promotion", "Spam", "Not Spam", "Not Spam", "Not Spam",
                                 "Spam", "Not Spam", "Promotion", "Not Spam", "Not
Spam",
                                 "Promotion", "Not Spam", "Not Spam", "Spam"]}
df = pd.DataFrame(email_data)
```

$Target : Spam$

```
email_spam = df[df['Target'] == 'Spam']
```

Email Size (KB)	Link Count	Unique Word Frequency (%)	Email Length (Chars)	Target
14.59	11	22.1	410	Spam
14.3	9	18.4	380	Spam
9.26	13	15.6	320	Spam

$Target : NotSpam$

```
email_notspam = df[df['Target'] == 'Not Spam']
```

Email Size (KB)	Link Count	Unique Word Frequency (%)	Email Length (Chars)	Target
28.24	1	82.4	850	Not Spam
32.62	2	79.8	920	Not Spam
23.83	1	88.5	710	Not Spam
32.90	3	75.2	980	Not Spam
22.65	2	84.1	690	Not Spam
27.71	1	81.3	820	Not Spam
22.67	2	86.4	680	Not Spam
26.21	1	83.2	790	Not Spam

$Target : Promotion$

```
email_promotion = df[df['Target'] == 'Promotion']
```

Email Size (KB)	Link Count	Unique Word Frequency (%)	Email Length (Chars)	Target
48.97	2	65.2	1240	Promotion
51.14	5	61.9	1310	Promotion
41.29	4	58.7	1150	Promotion

- **Step 2: Compute Mean and Variance**

Now we stop thinking in terms of rows, and start thinking in terms of *distribution*.

For every feature inside every class, we ask:

- Where is it centered? → **Mean (μ)**
- How much does it vary? → **Variance (σ^2)**

For each feature x_i and class y, we compute:

$$\mu_y = \frac{1}{n}\sum x_i; \quad \sigma_y^2 = \frac{1}{n}\sum (x_i - \mu_y)^2$$

We will let Python help with the calculation:

```
def mean_and_variance(features_df):
  mean_series = features_df.mean()
  variance_series = features_df.var()
  df = pd.DataFrame({"Mean": mean_series, "Variance": variance_series})
  return df

features = ['Email Size (KB)', 'Link Count', 'Unique Word Frequency (%)', 'Email Le
ngth (Chars)']
```

$Target : Spam$

```
# Mean and variance for each features Spam
X_spam = email_spam[features]
y_spam = email_spam['Target']
mean_and_variance(X_spam)
```

	Mean	Variance
Email Size (KB)	12.716667	8.982433
Link Count	11.000000	4.000000
Unique Word Frequency (%)	18.700000	10.630000
Email Length (Chars)	370.000000	2100.000000

$Target : NotSpam$

```
X_notspam = email_notspam[features]
y_notspam = email_notspam['Target']
mean_and_variance(X_notspam)
```

	Mean	Variance
Email Size (KB)	27.10375	16.670627
Link Count	1.62500	0.553571
Unique Word Frequency (%)	82.61250	16.598393
Email Length (Chars)	805.00000	12028.571429

$Target : Promotion$

```
X_promotion = email_promotion[features]
y_promotion = email_promotion['Target']
mean_and_variance(X_promotion)
```

	Mean	Variance
Email Size (KB)	47.133333	26.785633
Link Count	3.666667	2.333333
Unique Word Frequency (%)	61.933333	10.563333
Email Length (Chars)	1233.333333	6433.333333

Now pause for a moment. This right here is the entire model. No trees. No complex structure. No deep networks. Just numbers: Mean and variance.

And at this point we basically have everything we need.

- **Step 3: Apply Gaussian PDF**

Now comes the interesting part. We're going to classify a new email:

Email Size (KB)	Link Count	Unique Word Frequency (%)	Email Length (Chars)	Target
45.25	4	54.3	1000	?

Instead of jumping into all features at once. Let's slow down. Let's observe. Let's understand one piece first.

Start with just one feature: Say **Email Size = 45.25 KB** → And let's test it against the **Spam** class.

From earlier:

- $\mu(Spam, EmailSize) = 12.716667$
- $\sigma^2 = 8.982433 \rightarrow \sigma \approx \sqrt{8.982433}$

Now the question becomes: "*How likely is* 45.25*... if this email is Spam?*"

We use the Gaussian function:

$$P(x \mid y) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Now think about it before even calculating: Mean is around 12.7; Our value is 45.25

That's very far. Not slightly off. Not a little variation. Completely outside the normal range of Spam emails. So what happens? The exponent term becomes very large (negative). Which makes the entire probability **extremely close to 0**

No need for exact numbers yet. Just intuition. If Spam emails usually live around 12 KB... and suddenly you see something at 45 KB. Your model is already thinking: ***"Yeah... this doesn't feel like Spam".***

And this is the key difference. We're not matching exact values. We're asking: "How far is this from what I usually see?"

Now, we actually calculate it:

$$P(EmailSize = 45.25|Spam) = \frac{1}{8.982433\sqrt{2\pi}} e^{-\frac{(45.25-12.716667)^2}{2\times(8.982433)^2}}$$

Similarly, for the remaining features (once again, we will let Python do the heavy work):

```
from scipy.stats import norm

# New input
new_data = [45.25, 4, 54.3, 1000]
```

```
# Calculate PDF for Class Spam
pdf_spam = norm.pdf(new_data, X_spam.mean(), X_spam.std())
for i in range(len(pdf_spam)):
  print(f"P({features[i]}: {new_data[i]}| Spam) = {pdf_spam[i]}")
```

P(Email Size (KB): 45.25| Spam) = 3.446091340069849e-27
P(Link Count: 4| Spam) = 0.0004363413475228801
P(Unique Word Frequency (%): 54.3| Spam) = 1.5786991666158679e-27
P(Email Length (Chars): 1000| Spam) = 7.924500161558664e-44

$$P(x|Spam) = (3.446 \times 10^{-27})(0.000436)(1.578699 \times 10^{-27})(7.9245 \times 10^{-44}) = 1.8796 \times 10^{-100}$$

```
# Calculate PDF for Class Not Spam
pdf_notspam = norm.pdf(new_data, X_notspam.mean(), X_notspam.std())
for i in range(len(pdf_notspam)):
  print(f"P({features[i]}: {new_data[i]}| Not Spam) = {pdf_notspam[i]}")
```

P(Email Size (KB): 45.25| Not Spam) = 5.020366160604729e-06
P(Link Count: 4| Not Spam) = 0.00328622971510736
P(Unique Word Frequency (%): 54.3| Not Spam) = 3.191797155920671e-12
P(Email Length (Chars): 1000| Not Spam) = 0.0007487763645413115

$$P(x|NotSpam) = (5.020366 \times 10^{-6})(0.003286)(3.191797 \times 10^{-12})(0.000748776) = 3.942667 \times 10^{-23}$$

```
# Calculate PDF for Class Promotion
pdf_promotion = norm.pdf(new_data, X_promotion.mean(), X_promotion.std())
for i in range(len(pdf_promotion)):
  print(f"P({features[i]}: {new_data[i]}| Promotion) = {pdf_promotion[i]}")
```

P(Email Size (KB): 45.25| Promotion) = 0.07214471836332392
P(Link Count: 4| Promotion) = 0.2550241615316513
P(Unique Word Frequency (%): 54.3| Promotion) = 0.007784245607443229
P(Email Length (Chars): 1000| Promotion) = 7.227761551333146e-05

$$P(x|Promotion) = (0.0721447)(0.25502416)(0.0077842456)(7.22776 \times 10^{-5}) = 1.034156493 \times 10^{-8}$$

Since $max(P(x|Spam), P(x|NotSpam), P(x|Promotion)) = P(x|Promotion)$

→ The email is classified as $\boxed{Promotion}$

You might notice that the probabilities are tiny. And that is completely normal. Because remember what we just did: We didn't compute *one* probability. We multiplied **four probability densities together**. Each one is

already a small number. Multiply them and they shrink *fast*.

So values like: 10^{-23} or 10^{-100}. They look scary, but they don't mean anything is wrong.

What actually matters is this: **Relative comparison.** We don't care about the absolute value. We only care which one is *largest*. And here:

- Spam → 10^{-100} (basically impossible)
- Not Spam → 10^{-23} (still very small)
- Promotion → 10^{-8} (much larger in comparison)

So even though everything is tiny, $Promotion$ **dominates.** That's the decision.

> 💡 **A small but important note: Normalization**
> There's one more thing hiding here. Look at the features:
>
> - Email Size → tens
> - Link Count → small integers
> - Word Frequency → percentages
> - Email Length → hundreds or thousands
>
> Different scales. And Gaussian Naive Bayes *can* handle this but not always comfortably.
> Because variance depends on scale. Large-scale features (like Email Length) can dominate the calculation, not because they are more important but because their numbers are bigger.
> So in practice, we often apply **normalization (or standardization) to**:
>
> - Bring features to similar scale
> - Stabilize variance
> - Improve numerical behavior

It doesn't change the idea of the model. But it makes the model **behave better**.

So the probabilities are tiny. The math looks intimidating. But the decision? Still simple. ***Compare*** → ***Pick the largest*** → ***Move on.*** That's Gaussian Naive Bayes.

3. Gaussian Naive Bayes with Python

Here's how you would build a Gaussian Naive Bayes model in Python using the Scikit-Learn library. Using a toy Email Classification Dataset.

```
import pandas as pd

email_data = {"Email Size (KB)": [48.97, 14.59, 28.24, 32.62, 23.83, 14.30, 32.90,
51.14, 22.65, 27.71, 41.29, 22.67, 26.21, 9.26],
              "Link Count": [2, 11, 1, 2, 1, 9, 3, 5, 2, 1, 4, 2, 1, 13],
              "Unique Word Frequency (%)": [65.2, 22.1, 82.4, 79.8, 88.5, 18.4, 75.
2, 61.9, 84.1, 81.3, 58.7, 86.4, 83.2, 15.6],
              "Email Length (Chars)": [1240, 410, 850, 920, 710, 380, 980, 1310, 69
0, 820, 1150, 680, 790, 320],
              "Target": ["Promotion", "Spam", "Not Spam", "Not Spam", "Not Spam",
                                 "Spam", "Not Spam", "Promotion", "Not Spam", "Not
Spam",
```

```
                                    "Promotion", "Not Spam", "Not Spam", "Spam"]}
df = pd.DataFrame(email_data)
```

```
# Scale data
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Scale data
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df.drop("Target", axis=1))

# Target and feature
y = df["Target"]
X = df_scaled
# Spliting data
X_train, X_test, y_train, y_test = train_test_split(df_scaled, y, test_size=0.2, ra
ndom_state=42)
# Train the model
model = GaussianNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

```
# Calculate probabilities
y_proba = model.predict_proba(X_test)
for i, (pred, prob) in enumerate(zip(y_pred, y_proba)):
    print(f"Sample {i}; Probability: {prob}")
    print(f"+ Predicted: {pred}\n+ Actual Label: {y_test.iloc[i]}\n")
```

```
Sample 0; Probability: [1.00000000e+000 3.11652597e-057 4.01752816e-168]
+ Predicted: Not Spam
+ Actual Label: Not Spam
Sample 1; Probability: [1.00000000e+000 5.27675609e-078 5.02585212e-165]
+ Predicted: Not Spam
+ Actual Label: Not Spam
Sample 2; Probability: [8.45700410e-007 9.99999154e-001 5.03774356e-231]
+ Predicted: Promotion
+ Actual Label: Promotion
```

Summary

Gaussian Naive Bayes is still Naive Bayes. Still a classification model. Still built on probabilities and independence.

But instead of counting occurrences, it models **how data behaves**. It assumes that continuous features follow a **Gaussian distribution**, and uses **mean and variance** to describe each class.

No binning. No forced categories. Just letting the data stay continuous.

- It doesn't ask: "How many times did this happen?"
- It asks: *"How likely is this value... given what I usually see?"*

It may look simple. It may even look more naive than the original Naive Bayes model. But it does something very powerful: ***It turns raw numbers into probability.***

And that's enough to make decisions.

References

https://www.geeksforgeeks.org/machine-learning/gaussian-naive-bayes/

https://builtin.com/artificial-intelligence/gaussian-naive-bayes

https://www.youtube.com/watch?v=H3EjCKtlVog

Next Section:

20. Genetic Algorithms